

HelixSkills

HelixSkills

CLI AI سامانه مهارت‌های مبتنی بر حاکمیت و قانون اساسی برای عامل‌های

خلاصه

Helix است که زیرمجموعه CLI AI سامانه مهارت‌هایی برای عامل‌های HelixSkills را به ارث می‌برد، بنابراین تمامی قوانین حاکمیتی جهانی بدون قید و شرط اعمال Claude افزونه‌های کد، MCP می‌شوند. این سامانه مهارت‌های قابل نصب عامل‌ها، سرورهای ابزار و موتورهای قابل استفاده مجدداً در قالب یک کاتالوگ ثبت‌پذیر و مستند ارائه می‌دهد.

توضیح کوتاه

Helix است. این سامانه CLI AI سامانه مهارت‌هایی برای عامل‌های HelixSkills را به عنوان زیرمجموعه‌ای در خود جای داده تا تمامی قوانین جهانی اعمال شوند Constitution سپس مهارت‌های ثبت‌پذیر (پیشوند عمل، اعتبارسنج رسانه، چندمسیره، همگام‌سازی جلسه، چرخه و موتورهای قابل Claude دو افزونه کد، MCP حیات آیتم قابل انجام و موارد دیگر)، دو سرور ابزار استفاده مجدداً ارائه می‌دهد.

توضیح مفصل

CLI AI (پروانه آپاچی ۲.۰) سامانه مهارت‌هایی برای عامل‌های HelixSkills (مخزن skills) است که با وارونه‌سازی عمده‌ی ترتیب معمول آغاز می‌شود: حاکمیت در اولویت است و قابلیت‌ها در به ارث constitution/ را به عنوان زیرمجموعه Helix Constitution درجه دوم. این سامانه و constitution/CLAUDE.md می‌برد، بنابراین تمامی قوانین جهانی موجود در بدون قید و شرط اعمال می‌شوند — نه به عنوان عرفی که constitution/Constitution.md عملی ممکن است به آن پایبند باشد، بلکه به عنوان مجموعه قوانینی که به صورت فیزیکی در ساختار را به کار گیرد نمی‌تواند از قانون اساسی چشم‌پوشی HelixSkills پروژه گنجانده شده‌اند. عاملی که کند؛ قوانین همراه با کد منتقل می‌شوند.

فهرستی HelixSkills، در حالی که بیشتر «چارچوب‌های مهارتی» بر مفاهیم انتزاعی تکیه دارند ملموس و ثبت‌پذیر ارائه می‌دهد که می‌توان به آن اشاره کرد و آن را نصب نمود. هفت مهارت قانون action-prefix-system، media-validator، multitrack، reporting-workable-items، scheduled-work-queue، session-sync و workable-item-lifecycle — طیف گسترده‌ای از

مهارت‌های متوسط تا پیشرفته که از نام‌گذاری منظم اعمال گرفته تا اعتبارسنجی رسانه و چرخه کامل /حیات یک واحد کاری را پوشش می‌دهد. مهارت‌های پیش‌نویس اضافی (مروری بر اندروید، زبان جاوا نیز از پیش فهرست‌بندی و آماده فعال‌سازی شده‌اند. دو سرور ابزار (Linux سیستم عامل، Kotlin، Model این مهارت‌ها را از طریق (scheduled-work و MCP (media-validator Claude در اختیار عامل‌ها قرار می‌دهند، در حالی که دو افزونه کد Context Protocol همان قابلیت‌ها را مستقیماً در زمان اجرای عامل پیاده‌سازی (scheduled-work و helix) می‌کنند — مجموعه‌ای واحد از مهارت‌ها که از طریق هر سطحی که عامل با آن تعامل دارد، قابل دسترسی است.

پیاده‌سازی (continuum — در زیر کاتالوگ، چهار موتور قابل استفاده مجدد با عمق یک قرار دارند (در clickup_sync و token_optimizer، session_orchestrator شده)، به علاوه حال طراحی) — که زیرساخت مشترک مورد نیاز مهارت‌ها را فراهم می‌کنند تا از اختراع مجدد همان به تنهایی نمودار وابستگی صریحی را اعلام می‌کند token_optimizer. لوله‌کشی جلوگیری شود vasic-digital (TOON، Embeddings، VectorDB، که به بسته‌های اکوسیستم HelixDevelopment متعلق به LLMProvider و (Normalize، conversation) می‌رسد، بنابراین سیم‌کشی میان‌مخزنی آن قابل تمییزی است تا ضمناً تلویحی نباشد. در اطراف همه این‌ها مستندسازی منظمی وجود دارد: کاتالوگ مهارت‌ها، فهرست خودکار گراف مهارت‌ها، صفحات جزئیات هر مخزن و فهرست شفاف شکاف‌ها و ریسک‌ها که آنچه هنوز انجام نشده را مشخص می‌کند. و GitHub، GitLab، GitFlic و کل این سامانه برای تاب‌آوری و دسترسی منطقه‌ای در GitVerse آینه‌سازی شده است.

محتوا

چرا آن را ساختیم

به قابلیت‌هایی نیاز دارند که یکپارچه، تحت نظارت و قابل استفاده مجدد باشند AI و CLI عامل‌های برای این ساخته شد HelixSkills. نه اسکریپت‌های موقتی که هر بار قوانین را از نو ابداع کنند — که به عامل‌ها مجموعه‌ای بسته‌بندی‌شده و قابل ثبت از مهارت‌ها بدهد که به یک قانون اساسی مشترک متصل است، تا رفتار در هر عامل و پروژه‌ای که آن را به کار می‌گیرد، یکسان و قابل تمییزی بماند.

چرا انقلابی است

این سیستم قابلیت‌های عامل را از همان ابتدا قابل حمل و مطابق با قوانین می‌سازد، نه با تکیه بر انضباط فردی. هر مهارت یک واحد تحت نظارت، نسخه‌بندی‌شده و قابل نصب است که پشتوانه‌اش یک زیرمجموعه از قانون اساسی است — به این معنا که به محض ثبت یک مهارت توسط عامل، مجموعه قوانین استاندارد نیز به ارث می‌رسد، بدون هیچ فضایی برای انحراف. این امر چیزی را ممکن می‌سازد که پیش از این عملی نبود: انتقال یک قابلیت از یک عامل یا پروژه به دیگری با اطمینان از اینکه با و افزونه‌های کد MCP همان چارچوب نظارتی وارد می‌شود، و از طریق سطوح استاندارد (سرورهای در دسترس قرار می‌گیرد، نه انبوهی از اسکریپت‌های چسبیده سفارشی که هر بار قوانین را (Claude از نو تعریف می‌کنند.

نوآوری‌ها

- **Constitution** قوانین نظارتی جهانی به ارث می‌رسند، نه کپی: به عنوان زیرمجموعه می‌شوند — در ساختار درختی سوار می‌شوند تا هر عامل مصرف‌کننده به همان مجموعه قوانین استاندارد متصل باشد، و به روزرسانی‌ها از یک منبع واحد جریان یابند، نه از ده‌ها کپی منسوخ.

- و دوخته شده در فهرست گراف (register.sh) مهارت‌ها به عنوان واحدهای خودثبت‌شونده مهارت‌های خودکار: کاتالوگ همیشه قابل کشف باقی می‌ماند و هرگز با آنچه واقعاً نصب شده همگام نمی‌شود.
- و MCP مجموعه مهارت‌های یکسان از طریق سرورهای ابزار: در معرض قرارگیری چندسطحی به عامل‌ها می‌رسد — یک بار بنویسید، با هر محیط اجرایی که عامل Claude افزونه‌های کد استفاده می‌کند ارتباط برقرار کنید.
- (continuum, token_optimizer, session_orchestrator, clickup_sync) موتورهای قابل استفاده مجدد سطح اول که در سراسر اکوسیستم به اشتراک گذاشته می‌شوند، هر یک با اعلام وابستگی‌های صریح و قابل ممیزی بین مخازن، نه اتصال‌های پنهان.

بزرگ‌ترین چالش‌های فنی و راه‌حل‌های ما

- — حفظ رفتار یکسان و مطابق با قوانین عامل‌ها در میان مهارت‌ها و عامل‌های متعدد Helix پیاده‌سازی نظارت برای هر مهارت به مرور زمان باعث واگرایی می‌شود. راه‌حل این بود که constitution/ را به عنوان زیرمجموعه سوار کنیم تا قوانین موجود در constitution/Constitution.md و CLAUDE.md بدون قید و شرط اعمال شوند و از یک منبع بالادستی به‌روزرسانی شوند، نه اینکه کپی شوند و رها گردند.
- کاتالوگی که کسی نتواند — قابل نصب و کشف‌پذیر کردن مجموعه‌ای رو به رشد از مهارت‌ها آنچه در آن است را پیدا یا نصب کند، بی‌ارزش است. راه‌حل ما ثبت هر مهارت با INDEX در زمان نصب بود، به علاوه یک گراف مهارت‌های خودکار register.sh. مستندات جزئیات هر مخزن، تا کشف‌پذیری همیشه با واقعیت همگام باشد.
- نباید یک قابلیت برای — دسترسی به عامل‌هایی که با محیط‌های اجرایی مختلف کار می‌کنند هر میزبان از نو ساخته شود. راه‌حل این بود که یک مجموعه مهارت‌ها را هم در پشت تعریف‌های Claude (در constitution/mcp/) و هم افزونه‌های کد (در constitution/plugins/) بسته‌بندی کنیم، تا یک پیاده‌سازی واحد در سطوح مختلف در دسترس قرار گیرد.

محتوا

پشته فناوری

- انتخاب شده زیر ابزارهای نصب و ثبت باید در هر جایی که عامل — (زبان اصلی) Shell وجود دارد اجرا شوند، بدون نیاز به راه‌اندازی اولیه محیط‌زمان اجرا؛ این زبان اسکریپت‌های را قدرت می‌بخشد و مسیر ورود را بدون وابستگی و install_upstreams و register.sh قابل حمل نگه می‌دارد.
- Helix: انتخاب شده برای به ارث بردن حاکمیت بدون تکرار — زیرماترول‌های گیت به عنوان مرجع زنده سوار می‌شود، بنابراین constitution/ در مسیر Constitution به‌روزرسانی‌های قوانین از طریق یک اشاره‌گر منتشر می‌شوند، نه با کپی‌پیست و فراموشی.
- به عنوان رابط استاندارد و مستقل از — Model Context Protocol (MCP) MCP (media-validator و constitution/mcp/ در مسیر scheduled-work) محیط‌زمان اجرا برای عامل‌ها انتخاب شده است؛ دو سرور تعریف شده‌اند تا مهارت‌ها را به constitution/mcp/ در مسیر MCP عرضه می‌شوند که سطح constitution/plugins/ را برای میزبان دیگری بازتاب می‌دهند.
- انتخاب شده برای استخراج — (continuum, token_optimizer, session_orchestrator, clickup_sync) موتورهای قابل استفاده مجدد

ماشین‌آلات مشترک از مهارت‌های جداگانه به منظور استفاده متقابل در پروژه‌ها؛ به عنوان مثال token_optimizer به بسته‌های vasic-digital (TOON، Embeddings، VectorDB، Normalize، conversation) و HelixDevelopment's LLMProvider از طریق وابستگی‌های اعلام‌شده متصل می‌شود، نه با کد تکراری.

- انتخاب — (GitHub، GitLab، GitFlic، GitVerse) آینه‌سازی گیت چند میزبان شده تا قطع دسترسی در اثر خرابی یک میزبان یا مسدودسازی منطقه‌ای غیرممکن شود؛ همان مخزن در چهار فورژ مختلف به صورت زنده نگهداری می‌شود تا تاب‌آوری و دسترسی تضمین شود.

وضعیت و یادداشت‌های شفافیت

- و دو پلاگین عرضه شده‌اند؛ MCP هفت مهارت قانون اساسی، دو سرور وضعیت: بتا مهارت‌های پیش‌نویس فهرست شده و در انتظار فعال‌سازی هستند و سه موتور از چهار موتور سطح هنوز در (session_orchestrator، token_optimizer، clickup_sync) اول مرحله طراحی قرار دارند.
- معرفی می‌کند؛ مسیر استاندارد helix_skills این پروژه را با نام README فایل است. شماره یافته‌های ثبت‌شده در HelixDevelopment/skills برابر با GitHub بر اساس گزارش خود پروژه است README

اصلی-Helix: اولویت